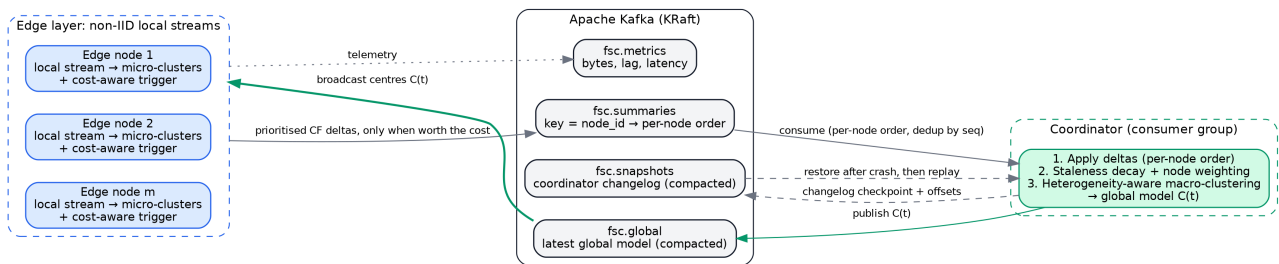


FedCAST

Budget-Aware Federated Stream Clustering over Apache Kafka under Non-IID Data and Network Constraints

Final project report: requirements, what was built, results, design rationale and next steps

Course: Stream Processing Analytics · Repository: github.com/kushsriv/spa · 27 September 2026



System architecture: edge nodes, Kafka topics, coordinator

Headline results

- **10-61% fewer bytes** than the best baseline to reach within 5% of centralised quality, and up to **8.5× fewer** than periodic sync
- **Ranked first** at matched budgets (Friedman $p = 2.1e-04$; mean rank 1.1 vs 2.5 for the next best)
- **2.5× faster** detection of newly emerging clusters than periodic updates, while sending less
- Coordinator killed with SIGKILL: state restored from Kafka in **1.1 s, identical** to a coordinator that never crashed
- Broker offline for 34 s: **0 updates lost** (231/231 applied)
- Whole system runs in **under 1 GB of RAM** on a laptop, with or without Docker

Contents

- 1. Your requirements and how they were met
- 2. What was built
- 3. How it works (method and flowcharts)
- 4. Results
- 5. Why this tech stack and these features (alternatives compared)
- 6. How to run it
- 7. The paper and where to submit
- 8. What your team must do before submission
- 9. Honest limitations

1. Your requirements and how they were met

Your requirement	Status	Where
Topic: cost-aware federated stream clustering, non-IID data, network constraints	Done: FedCAST method, system and evaluation	src/fedcast, paper/
Kafka is essential	Kafka's ordering, idempotence, replay and compaction are part of the design; tested under crashes	kafka_runtime.py, §4.4
Q1-level research paper	Full manuscript in the Elsevier template with theory (3 propositions and proofs), 2,000+ runs and statistics	paper/main.pdf
Extensive research: problem, objectives, methodology, outcomes	Done	docs/00_research_proposal.md
Claude writes all the code	About 2,500 lines of Python, tests, experiment and analysis scripts	src/, experiments/, tests/
Deadline: as if due today	Everything built, run, analysed and written up in one session	this report
Laptops with under 4 GB free RAM	Whole system under 1 GB, measured; runs without Docker	docs/02_low_memory_setup.md
Target IEEE, ACM, Elsevier, Springer	Venue shortlist; manuscript formatted for Elsevier FGCS	§7
Explain the tech stack, features and alternatives	Detailed rationale document	docs/03_design_rationale.md, §5

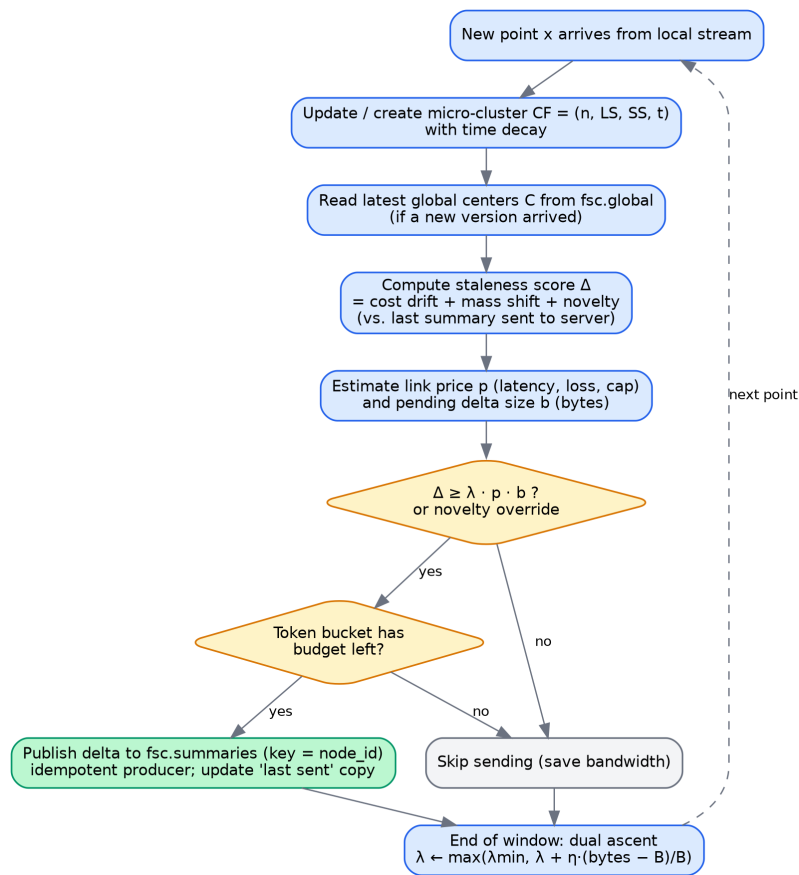
2. What was built

Component	What it does
Edge node (node.py, microcluster.py)	Online micro-clustering with time-decayed cluster features; keeps an exact copy of what the server holds
FedCAST send policy (policies.py)	Staleness score linked to the clustering objective; sends only the most valuable micro-clusters; dual-ascent threshold; token-bucket budget; link price; novelty override
Coordinator (coordinator.py, macro.py)	Applies each update exactly once; decays stale data; weighted k-means with swap search to protect single-node clusters
Kafka runtime (kafka_runtime.py)	Real edge and coordinator processes; changelog-based crash recovery; global-model broadcast
Wire format (codec.py)	Compact binary messages whose size is known before sending
Simulator and network emulator (sim.py, netem.py)	Deterministic, seeded, the same code as the real system; delay, loss, bandwidth, outages
Baselines	Centralised-Raw, Naive, Periodic, Periodic- Δ , Change-Threshold (Tran/DGClust), Norm-Trigger (EventGraD), k-FED (ICML 2021)
Datasets (data.py)	SynDrift (evolving, ours), NSL-KDD, Shuttle, Pen-Digits, Letter; five non-IID split types
Experiments	E1-E8 (2,000+ simulated runs, 5 seeds) and K1-K4 on a real Kafka broker
Tests	Unit tests, including numerical checks of the propositions and the budget guarantee
CLI	fedcast sim demo edges coordinator reset-topics
Paper	Complete manuscript with proofs, figures, tables and statistics (paper/main.pdf)

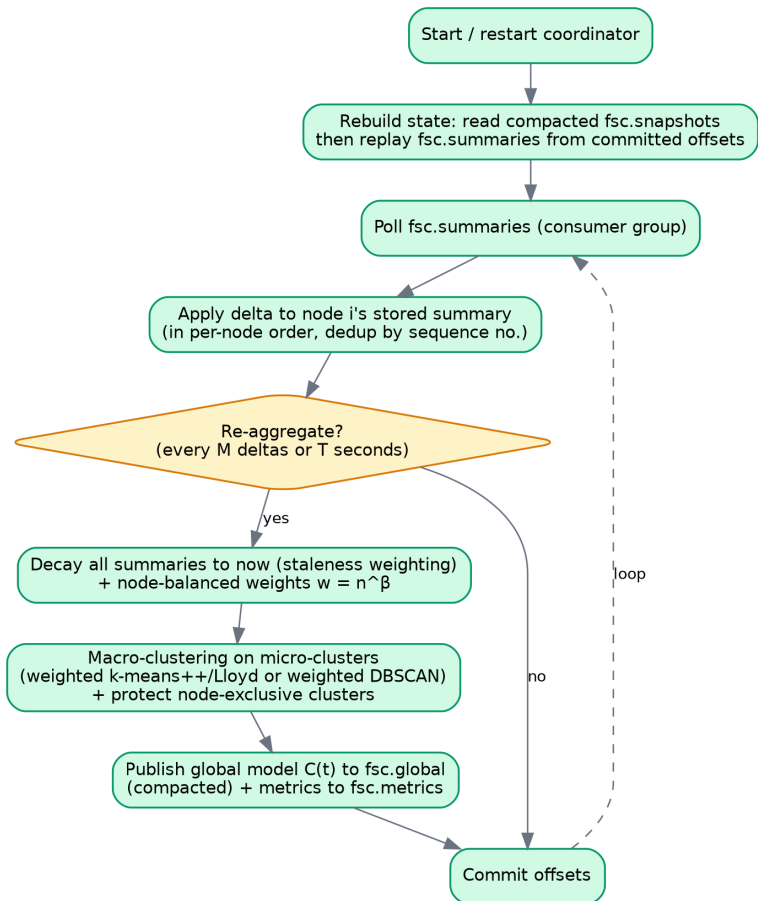
3. How it works

Every edge node asks at each tick: is sending now worth its network cost, and which part should I send?

- **Staleness score.** For every micro-cluster, the node computes how much the server's copy distorts the global k-means cost, using a closed-form formula and the global model it receives back. The sum provably bounds the server's error (Propositions 1 and 2).
- **Prioritisation.** Only the micro-clusters carrying 90% of that staleness are sent. This alone is the biggest win in the ablation study.
- **Budget.** Send if value $\geq \lambda \times \text{link price} \times \text{bytes}$. λ adapts automatically to hit the byte budget, and a token bucket makes the budget a hard guarantee (Proposition 3).
- **Novelty.** A cluster the server has never seen is sent immediately.
- **Coordinator.** Applies updates exactly once (per-node sequence numbers), decays stale data, and re-clusters with warm-started k-means plus swap search.



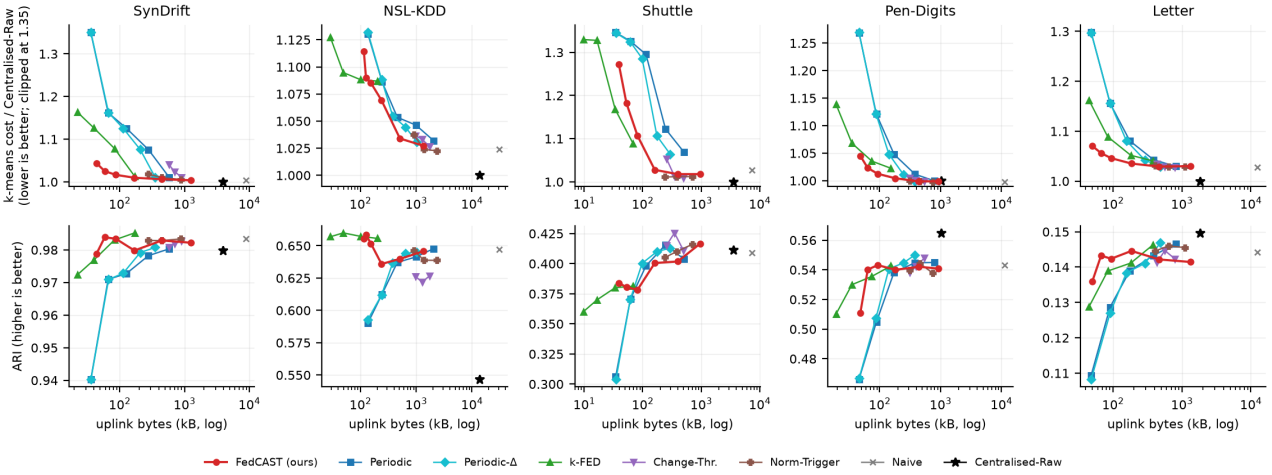
Edge node decision flowchart



Coordinator flowchart (with crash recovery from the Kafka changelog)

4. Results

4.1 Communication versus quality (RQ1)

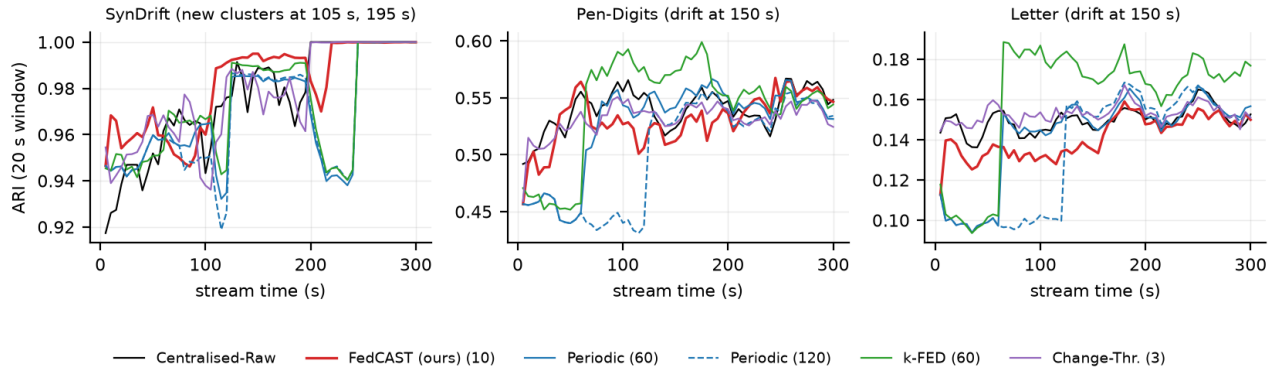


Bytes (kB) each method needs to get within 5% of the centralised learner's cost:

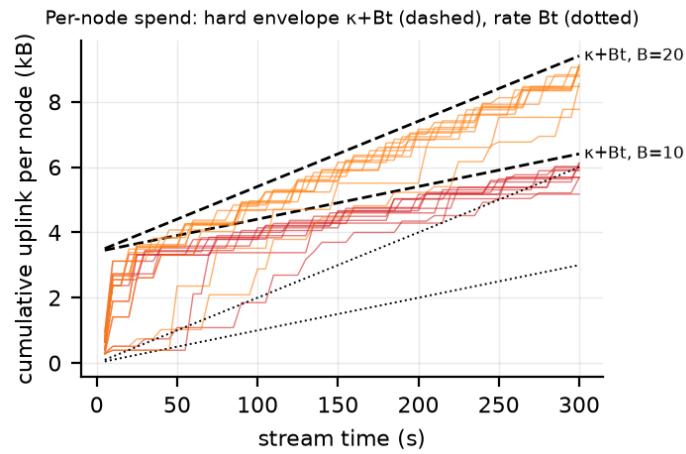
Dataset	FedCAST (ours)	Periodic	Periodic- Δ	k-FED	Change-T hr.	Norm-Trigger	saving vs best baseline
SynDrift	44	372	256	113	574	276	61%
NSL-KDD	362	668	484	—	969	907	25%
Shuttle	133	—	—	—	263	242	45%
Pen-Digits	48	169	140	53	309	309	10%
Letter	83	334	256	204	435	412	60%

Statistics. Friedman $\chi^2 = 19.6$, $p = 2.1e-04$ over 10 dataset**x**budget blocks. Holm-corrected Wilcoxon: FedCAST beats Periodic ($p = 4e-13$, wins 96%), Periodic- Δ ($p = 5e-11$, 94%) and k-FED ($p = 2e-09$, 88%).

4.2 Drift and newly emerging clusters (RQ3)

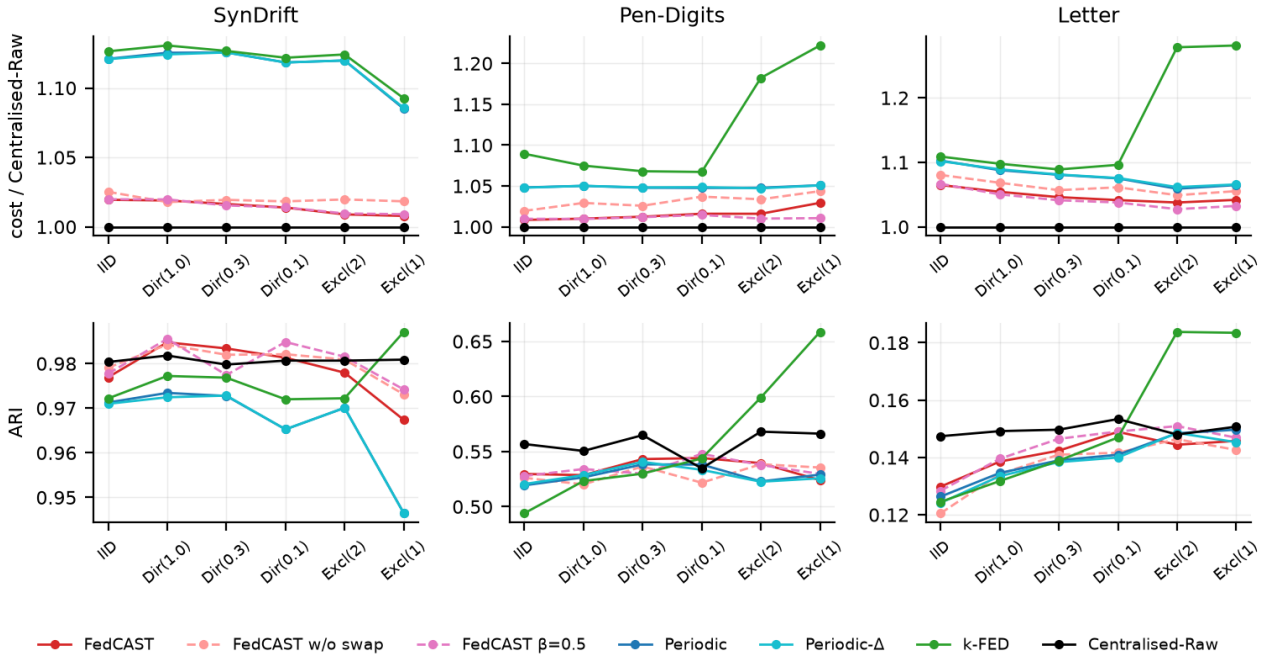


ARI over time. On SynDrift, new clusters appear at 105 s and 195 s; FedCAST reports them about 2.5× faster than Periodic and k-FED while sending less.



Every node stays under the hard budget envelope; after bootstrap the slope equals the budget.

4.3 Non-IID data (RQ2)



From IID to cluster-exclusive nodes: FedCAST has the lowest cost in all 18 settings.

4.4 Ablation (cost ratio, lower is better)

Variant	SynDrift	NSL-KDD	Shuttle	Pen-Digits	Letter	mean kB	budget use
fedcast	1.016	1.085	1.107	1.012	1.046	102.1	1.70
no-dual	1.019	1.088	1.108	1.011	1.047	101.5	1.69
no-price	1.017	1.090	1.065	1.012	1.044	102.7	1.71
no-novelty	1.017	1.090	1.092	1.012	1.047	101.9	1.70
no-priority	1.333	1.132	1.331	1.035	1.061	87.3	1.45
full-summary	1.295	1.132	1.321	1.037	1.054	89.2	1.49
no-swap	1.019	1.101	1.218	1.026	1.057	102.0	1.70
no-decay	1.017	1.089	1.106	1.011	1.047	102.0	1.70

4.5 Real Kafka: correctness, fault tolerance, scale (RQ4, RQ5)

Byte accounting: our estimate vs Kafka's own producer statistics

Method	Our estimate (B)	Kafka txmsg_bytes (B)	Difference
fedcast 20	55,978	54,340	+3.0%
fedcast 150	230,798	226,028	+2.1%
periodic 30	189,160	187,900	+0.7%
pdelta 30	153,264	152,004	+0.8%
kfed 30	57,760	56,500	+2.2%
raw 0	2,486,473	2,450,275	+1.5%

Test	Result
Coordinator SIGKILL, restarted after 8 s	State restored from the compacted changelog in 1.08 s; identical to the shadow coordinator: yes; mass difference 0
Broker stopped for 34 s mid-run	231/231 updates applied, 0 lost, 0 delivery errors

Scalability on one machine (real broker)

Nodes	Method	Uplink kB	End-to-end p50 / p95 (ms)	Edge / coordinator / broker RSS (MB)
5	fedcast	32	35 / 57	147 / 141 / 475
5	raw	2,458	179 / 355	154 / 142 / 475
10	fedcast	65	33 / 58	156 / 141 / 475
10	raw	2,486	156 / 356	161 / 141 / 475
20	fedcast	132	35 / 57	164 / 141 / 475
20	raw	2,572	171 / 393	165 / 142 / 475
50	fedcast	325	33 / 57	172 / 141 / 475
50	raw	2,815	157 / 283	180 / 142 / 471

5. Why this tech stack and these features

Choice	Why it is the best fit	Alternatives and why not
Apache Kafka (KRaft)	Per-key order, idempotent producers, replayable log, log compaction: exactly what federated deltas and crash recovery need; runs in about 400 MB	MQTT/RabbitMQ: no replay or compaction. Pulsar: needs BookKeeper, too heavy. Redpanda: not Apache Kafka. Cloud queues: paid, not reproducible.
Python services + confluent-kafka	About 150 MB per process; full control over when to send; librdkafka is fast and exposes byte statistics	Flink/Spark: 1.5–2 GB+ RAM, dataflow model does not fit per-device decisions. kafka-python: slower, fewer stats.
CF micro-clusters	Exactly additive; decay keeps centroids; closed-form k-means cost enables the objective-linked trigger	Grids: blow up in high dimensions. Coresets: no cheap deltas. Sketches: not geometric. k-FED centres: lose shape and size.
Objective-linked, prioritised trigger	Bounds the server's error; spends bytes where the objective changes	Periodic: blind to change. Thresholds: retune per dataset. Norm triggers: not tied to the clustering objective.
Exponentiated dual ascent + token bucket	Scale-free (one η for all datasets), provable; hard guarantee from the bucket	PID: three gains per dataset. Fixed λ : breaks when data rates change.
Swap local search	Principled k-means local search; protects single-node clusters	Plain warm-start Lloyd: absorbs new clusters. FedAvg of centres: label-permutation problem.
Fixed binary wire format	Exact size known before sending; zero-copy; no schema registry	JSON: 3–5× larger. Avro/Protobuf: need a registry service.
Simulator + real Kafka, one code base	Thousands of seeded runs plus real systems measurements; bytes validated against Kafka	Only real runs: too slow for statistics. Only simulation: not credible for systems claims.

Full discussion: docs/03_design_rationale.md

6. How to run it

```
python -m venv .venv && source .venv/bin/activate
pip install -e .
scripts/kafka_native.sh start # or: docker compose -f docker/docker-compose.yml up -d
fedcast demo --dataset syndrift --method fedcast --param 20 --speed 10

# reproduce everything
python -m pytest
python experiments/run.py E1 E2 E3 E4 E5 E6 E7 E8
python experiments/kafka_experiments.py
python experiments/analyze.py
cd paper && latexmk -pdf main.tex
```

7. The paper and where to submit

- **Manuscript:** paper/main.pdf (Elsevier elsarticle template), with abstract, introduction, related work, formulation, method with three propositions (proofs in the appendix), Kafka system design, experimental setup, results for RQ1–RQ5, ablations, discussion and limitations.
- **First choice:** Elsevier Future Generation Computer Systems (Kafka-ML appeared there), or the IEEE Internet of Things Journal. Also a good fit: IEEE TKDE (theory-heavy version), Springer DMKD, ACM TKDD. Conference option: ACM DEBS.
- Switching to IEEE: change the document class to IEEEtran; the section files are template-agnostic.

8. What your team must do before submission

#	Task
1	Put real author names, affiliations and emails in paper/main.tex; agree authorship with your professor
2	Check every reference against the original paper (some metadata came from search snippets)
3	Check the target journal's current quartile, scope and article-processing charges; read its guide for authors
4	Read the whole paper critically; rerun the demo and the tests on your own laptops
5	Optional, strengthens the paper: run the Kafka experiments on 2–3 real laptops over Wi-Fi
6	Write the cover letter, suggest reviewers, and submit

9. Honest limitations

- At the very tightest budgets (about 1% of raw traffic), k-FED is cheaper: FedCAST's first full summary costs more than that.
- With one or two well-separated classes per node, k-FED reaches higher label agreement (ARI), though a worse k-means cost.
- Link price, novelty and dual ascent barely change average cost; they matter for wire cost, reaction time and loose budgets respectively.
- Scale was tested up to 50 nodes on one machine; real wide-area deployments are future work.
- Summaries are aggregates, not differentially private; adding DP noise is future work.